

Linjär intervall:

a) Time complexity of each operation

- **insert(i):**
 - Just appends at end → $O(1)$
 - Also sets sorted = false → constant time Answer: $O(1)$
- **find(x):**
 - If array not sorted → $O(n \log n)$ (Heapsort) + $O(\log n)$ (binary search)
 - If already sorted → just $O(\log n)$
 - Answer: Worst case (unsorted): $O(n \log n)$ Else: $O(\log n)$

(b) First insert all n intervals, then call find n times

- The n inserts each cost **$O(1)$** → total **$O(n)$**
- After all inserts, sorted = false, so the **first find** must sort → **$O(n \log n)$**
- Remaining $(n - 1)$ finds → each **$O(\log n)$** **Total time:**
 - $O(n) + O(n \log n) + O(n \log n) = O(n \log n)$

(c) Alternate case — after each insert, we immediately call find

- That means the pattern is: insert, find, insert, find, ... (n times total)
- Each sort costs **$O(k \log k)$** , where k is the current number of intervals ($\approx n$ by the end).
- So total cost $\approx O(1 \log 1) + O(2 \log 2) + O(3 \log 3) + \dots + O(n \log n) \approx \mathbf{O(n^2 \log n)}$

Värmeberäkningar:

(a) Algorithm design and data structures

- **Input:**
 - A sorted list of temperature readings (date, time, temp)
 - **Output:**
 - List of heatwave intervals, e.g., [(2025-06-01, 2025-06-10)..]
 - **Data structures:** Use a dictionary or map to store:
 - key = date
 - value = list of temperatures that day
1. **Create a function find_heatwaves(measurements)**
 - **Input:** a list of temperature measurements with (date, time, temperature)
 - **Output:** list of heatwave intervals (start_date, end_date)
 2. **Create a dictionary daily_max**
 - Key: date
 - Value: the highest temperature recorded that day
 3. **For each measurement in the data:**
 - If the date is not in daily_max, add it with the current temperature.
 - Otherwise, compare with the existing value and store the maximum temperature for that date.
 4. **Sort the dates in chronological order (since we'll check consecutive days).**

5. Initialize:

- start_day = none (marks the start of a potential heatwave)
- count = 0 (number of consecutive hot days)
- result = empty list to store found intervals.

6. For each day in sorted order:

- If daily_max[day] $\geq 25^{\circ}\text{C}$:
 - If start_day is empty $\rightarrow$ mark the current day as the start of the heatwave.
 - Increase count by 1.
- Else (temperature $< 25^{\circ}\text{C}$):
 - If count ≥ 5 , record (start_day, previous_day) as a heatwave.
 - Reset count = 0 and clear start_day.

7. After the loop, if the final streak had ≥ 5 days, add that interval too.

8. Return or print all heatwave intervals.

Positiv summering

a) Complexity for Alternative 1

- Insertion sort has a best case of $O(n)$ if all elements are already sorted, or if only one (or a few) elements are out of place. Its worst and average case is $O(n^2)$.
- So if the list x is already sorted:
 - **Step 0** (storing the list) takes $O(n)$ time.
 - **Step 1** (sorting with insertion sort) takes $O(n)$ time.
 - **Step 2** (removing the last two elements) takes $O(1)$.
 - **Step 3** (adding the new element) also takes $O(1)$, since the dynamic array doesn't usually need to resize.
 - $\rightarrow$ One iteration total: $O(n)$. So total **best case** = $O(n^2)$.
- This process repeats $n-1$ times, since the array gets shorter by one each time.
- **If the list is not sorted from the start (worst case):**
 - The first insertion sort takes $O(n^2)$ time.
 - The rest of the analysis is the same: later sorts are much faster, $O(n)$ each.
- So, the total worst-case scenario is $O(n^2)$ as well.

(b) Alternative 2:

Step 1 (remove the smallest element) takes $O(\log n)$. Same complexity for 2 and 3.

Steps 1–3 are repeated n times, since the heap shrinks by one each time.

- There *is* a best-case scenario if the removed or inserted element doesn't need to be repositioned in the heap; those steps could take $O(1)$, but this is extremely rare.
- So we assume the best and worst cases are the same.
 - Each loop iteration = $O(\log n)$
 - Done n times $\rightarrow$ **total** = $O(n \log n)$.

for($i = 0$; $i < n$; $i++$) $\rightarrow O(n)$

for($i = 1$; $i < n$; $i *= 2$) $\rightarrow O(\log n)$

for($i = n$; $i > 0$; $i /= 2$) $\rightarrow O(\log n)$

